

NYC 311 Data Curation Project ITAI 3375 — Curating Data for AI Optimization

A repeatable subset of NYC 311 data Profiles of raw and edited data

A pipeline for documented curation Location handling that respects privacy Initial urgency labels for human evaluation Five quality scores before and after A quality chart that is ready for presentation 50-record annotation review sheet and curated CSV

Important: The urgency markings are not agency or emergency judgments, but rather preliminary guidelines. Since external ground truth is necessary for actual accuracy, the accuracy result is clearly marked as an internal logic proxy.

```
1 !pip -q install pandas numpy matplotlib requests
```

✓ Load the corrected functions

```
1 # NYC 311: Duplicate and Cleanliness Analysis
2 # ITAI 3375
3 #
4 # This corrected version visibly reports:
5 # 1. Exact duplicate rows
6 # 2. Duplicate Unique Keys
7 # 3. Potential duplicate records for review
8 # 4. Missing values and cleanliness before/after
9 # 5. A cleaning audit log showing exactly what changed
10
11 import math
12 import time
13 from pathlib import Path
14
15 import matplotlib.pyplot as plt
16 import numpy as np
17 import pandas as pd
18 import requests
19
20 DATASET_ID = "erm2-nwe9"
21 API_URL = f"https://data.cityofnewyork.us/resource/{DATASET_ID}.json"
22
23 TARGET_ROWS = 100_000      # Change to the exact number used in your project
24 PAGE_SIZE = 25_000
25
26 OUTPUT_DIR = Path("nyc311_outputs")
27 OUTPUT_DIR.mkdir(exist_ok=True)
28
29 COLUMNS = [
30     "unique_key", "created_date", "closed_date", "agency",
31     "complaint_type", "descriptor", "location_type",
32     "incident_zip", "borough", "status", "due_date",
33     "resolution_description"
```

```

34 ]
35
36 TEXT_COLUMNS = [
37     "agency", "complaint_type", "descriptor", "location_type",
38     "borough", "status", "resolution_description"
39 ]
40
41
42 def download_data(target_rows=TARGET_ROWS):
43     """Download a reproducible subset from NYC Open Data."""
44     frames = []
45     pages = math.ceil(target_rows / PAGE_SIZE)
46
47     for page in range(pages):
48         limit = min(PAGE_SIZE, target_rows - page * PAGE_SIZE)
49         params = {
50             "$select": ",".join(COLUMNS),
51             "$limit": limit,
52             "$offset": page * PAGE_SIZE,
53             "$order": "created_date DESC, unique_key DESC",
54         }
55
56         response = requests.get(API_URL, params=params, timeout=90)
57         response.raise_for_status()
58         records = response.json()
59
60         if not records:
61             break
62
63         frames.append(pd.DataFrame(records))
64         print(f"Downloaded page {page + 1}: {len(records):,} rows")
65         time.sleep(0.1)
66
67     if not frames:
68         raise RuntimeError("No records were returned.")
69
70     df = pd.concat(frames, ignore_index=True)
71
72     for column in COLUMNS:
73         if column not in df.columns:
74             df[column] = pd.NA
75
76     return df[COLUMNS].head(target_rows)
77
78
79 def duplicate_report(df):
80     """Return visible counts for three duplicate checks."""
81     exact_count = int(df.duplicated(keep=False).sum())
82
83     keys = df["unique_key"].astype("string").str.strip()
84     duplicate_key_count = int(
85         (keys.notna() & keys.duplicated(keep=False)).sum()
86     )
87
88     fingerprint_columns = [
89         "created_date", "agency", "complaint_type",
90         "descriptor", "incident_zip", "borough"
91     ]

```

```

92
93     fingerprint = (
94         df[fingerprint_columns]
95         .fillna("")
96         .astype("string")
97         .apply(lambda col: col.str.strip().str.lower())
98     )
99
100     potential_count = int(
101         fingerprint.duplicated(keep=False).sum()
102     )
103
104     return pd.DataFrame({
105         "Duplicate Check": [
106             "Exact duplicate rows",
107             "Rows with duplicate Unique Keys",
108             "Potential duplicate rows for review"
109         ],
110         "Count": [
111             exact_count,
112             duplicate_key_count,
113             potential_count
114         ],
115         "Meaning": [
116             "All selected fields are identical",
117             "The same service-request ID appears more than once",
118             "Same date, agency, complaint, descriptor, ZIP, and borough"
119         ]
120     })
121
122
123 def duplicate_examples(df, max_rows=20):
124     """Return examples for each duplicate check."""
125     exact = df[df.duplicated(keep=False)].head(max_rows)
126
127     keys = df["unique_key"].astype("string").str.strip()
128     duplicate_keys = df[
129         keys.notna() & keys.duplicated(keep=False)
130     ].sort_values("unique_key").head(max_rows)
131
132     fingerprint_columns = [
133         "created_date", "agency", "complaint_type",
134         "descriptor", "incident_zip", "borough"
135     ]
136
137     fingerprint = (
138         df[fingerprint_columns]
139         .fillna("")
140         .astype("string")
141         .apply(lambda col: col.str.strip().str.lower())
142     )
143
144     potential = df[
145         fingerprint.duplicated(keep=False)
146     ].sort_values(fingerprint_columns).head(max_rows)
147
148     return exact, duplicate_keys, potential

```

```

149
150
151 def cleanliness_report(df, stage):
152     """
153     Cleanliness = percentage of values that are present.
154     Empty strings are treated as missing.
155     """
156     work = df.replace(r"^\\s*$", np.nan, regex=True)
157
158     return pd.DataFrame({
159         "Stage": stage,
160         "Column": work.columns,
161         "Total Rows": len(work),
162         "Missing Values": [int(work[c].isna().sum()) for c in work.columns],
163         "Missing Percent": [
164             round(work[c].isna().mean() * 100, 2) for c in work.columns
165         ],
166         "Present Values": [int(work[c].notna().sum()) for c in work.columns],
167         "Cleanliness Percent": [
168             round(work[c].notna().mean() * 100, 2) for c in work.columns
169         ]
170     })
171
172
173 def overall_cleanliness(df):
174     work = df.replace(r"^\\s*$", np.nan, regex=True)
175     return round(work.notna().mean().mean() * 100, 2)
176
177
178 def clean_data(raw):
179     """Clean the dataset and record every action."""
180     df = raw.copy()
181     audit = []
182
183     def record(step, before, after, details):
184         audit.append({
185             "Cleaning Step": step,
186             "Rows Before": before,
187             "Rows After": after,
188             "Rows Removed": before - after,
189             "Details": details
190         })
191
192     before = len(df)
193     df = df.replace(
194         ["", " ", "N/A", "NA", "NULL", "null", "None"],
195         pd.NA
196     )
197     record(
198         "Standardize missing values", before, len(df),
199         "Converted blank and placeholder values to missing"
200     )
201
202     before = len(df)
203     for column in TEXT_COLUMNS:
204         df[column] = (
205             df[column].astype("string")
206             .str.strip()

```

```

207         .str.replace(r"\s+", " ", regex=True)
208     )
209     record(
210         "Normalize text", before, len(df),
211         "Trimmed spaces and removed repeated whitespace"
212     )
213
214     before = len(df)
215     df = df.drop_duplicates(keep="first")
216     record(
217         "Remove exact duplicate rows", before, len(df),
218         "Removed records where every selected field matched"
219     )
220
221     before = len(df)
222     df["unique_key"] = df["unique_key"].astype("string").str.strip()
223     df = df.drop_duplicates(subset=["unique_key"], keep="first")
224     record(
225         "Remove duplicate Unique Keys", before, len(df),
226         "Kept one row for each service-request ID"
227     )
228
229     before = len(df)
230     valid_key = df["unique_key"].str.match(r"^\d+$", na=False)
231     df = df[valid_key].copy()
232     record(
233         "Remove invalid Unique Keys", before, len(df),
234         "Kept numeric service-request IDs only"
235     )
236
237     before = len(df)
238     df["agency"] = df["agency"].str.upper()
239     df["borough"] = df["borough"].str.upper().fillna("UNSPECIFIED")
240     df["status"] = df["status"].str.upper().fillna("UNSPECIFIED")
241     record(
242         "Standardize categories", before, len(df),
243         "Standardized agency, borough, and status"
244     )
245
246     before = len(df)
247     df["descriptor"] = df["descriptor"].fillna("Not Provided")
248     df["location_type"] = df["location_type"].fillna("Not Specified")
249     df["resolution_description"] = (
250         df["resolution_description"].fillna("Not Available")
251     )
252     record(
253         "Label selected missing descriptions", before, len(df),
254         "Used transparent labels instead of inventing values"
255     )
256
257     before = len(df)
258     for column in ["created_date", "closed_date", "due_date"]:
259         df[column] = pd.to_datetime(df[column], errors="coerce")
260     record(
261         "Parse dates", before, len(df),
262         "Converted dates; invalid dates became missing"
263     )
264

```

```

265     before = len(df)
266     invalid_closed = (
267         df["created_date"].notna()
268         & df["closed_date"].notna()
269         & (df["closed_date"] < df["created_date"])
270     )
271     invalid_count = int(invalid_closed.sum())
272     df.loc[invalid_closed, "closed_date"] = pd.NaT
273     record(
274         "Fix impossible date order", before, len(df),
275         f"Set {invalid_count:,} impossible closed dates to missing"
276     )
277
278     before = len(df)
279     zip5 = (
280         df["incident_zip"].astype("string")
281         .str.extract(r"(\d{5})", expand=False)
282     )
283     df["zip3_generalized"] = zip5.str[:3].fillna("Unknown")
284     df = df.drop(columns=["incident_zip"])
285     record(
286         "Generalize ZIP code", before, len(df),
287         "Retained first three digits and removed full ZIP"
288     )
289
290     return df.reset_index(drop=True), pd.DataFrame(audit)
291
292
293 def compare_cleanliness(before_report, after_report):
294     before = before_report[
295         ["Column", "Missing Values", "Cleanliness Percent"]
296     ].rename(columns={
297         "Missing Values": "Missing Before",
298         "Cleanliness Percent": "Cleanliness Before (%)"
299     })
300
301     after = after_report[
302         ["Column", "Missing Values", "Cleanliness Percent"]
303     ].rename(columns={
304         "Missing Values": "Missing After",
305         "Cleanliness Percent": "Cleanliness After (%)"
306     })
307
308     result = before.merge(after, on="Column", how="outer")
309     result["Missing Reduced"] = (
310         result["Missing Before"].fillna(0)
311         - result["Missing After"].fillna(0)
312     )
313     result["Cleanliness Change"] = (
314         result["Cleanliness After (%)"].fillna(0)
315         - result["Cleanliness Before (%)"].fillna(0)
316     ).round(2)
317
318     return result
319
320
321 def plot_cleanliness(comparison):
322     chart = comparison.dropna(subset=["

```

```

322     chart = chart.plot(
323         "Cleanliness Before (%)",
324         "Cleanliness After (%)"
325     ])
326
327     ax = chart.plot(
328         x="Column",
329         y=["Cleanliness Before (%)", "Cleanliness After (%)"],
330         kind="bar",
331         figsize=(13, 6)
332     )
333     ax.set_title("NYC 311 Cleanliness Before and After Curation")
334     ax.set_xlabel("Column")
335     ax.set_ylabel("Cleanliness (%)")
336     ax.set_ylim(0, 105)
337     ax.tick_params(axis="x", rotation=45)
338     plt.tight_layout()
339     plt.savefig(
340         OUTPUT_DIR / "cleanliness_before_after.png",
341         dpi=200,
342         bbox_inches="tight"
343     )
344     plt.show()
345
346
347 def plot_duplicates(before_report, after_report):
348     chart = before_report[["Duplicate Check", "Count"]].rename(
349         columns={"Count": "Before"}
350     ).merge(
351         after_report[["Duplicate Check", "Count"]].rename(
352             columns={"Count": "After"}
353         ),
354         on="Duplicate Check",
355         how="outer"
356     )
357
358     ax = chart.plot(
359         x="Duplicate Check",
360         y=["Before", "After"],
361         kind="bar",
362         figsize=(11, 6)
363     )
364     ax.set_title("Duplicate Checks Before and After Cleaning")
365     ax.set_xlabel("")
366     ax.set_ylabel("Rows")
367     ax.tick_params(axis="x", rotation=20)
368     plt.tight_layout()
369     plt.savefig(
370         OUTPUT_DIR / "duplicates_before_after.png",
371         dpi=200,
372         bbox_inches="tight"
373     )
374     plt.show()
375
376
377 def run_project():
378     raw = download_data()
379

```

```

380     print("\nRAW DATA SHAPE:", raw.shape)
381     display(raw.head(10))
382
383     duplicates_before = duplicate_report(raw)
384     print("\nDUPLICATES BEFORE CLEANING")
385     display(duplicates_before)
386
387     exact, duplicate_keys, potential = duplicate_examples(raw)
388     print("Exact duplicate examples")
389     display(exact)
390     print("Duplicate Unique Key examples")
391     display(duplicate_keys)
392     print("Potential duplicate examples")
393     display(potential)
394
395     before_cleanliness = cleanliness_report(raw, "Before Cleaning")
396     print("Overall cleanliness before:", overall_cleanliness(raw), "%")
397     display(before_cleanliness)
398
399     cleaned, audit = clean_data(raw)
400     print("\nCLEANING AUDIT LOG")
401     display(audit)
402
403     # Add a temporary missing incident_zip column only so the same
404     # duplicate-report function can compare before and after.
405     cleaned_for_duplicate_check = cleaned.copy()
406     cleaned_for_duplicate_check["incident_zip"] = pd.NA
407
408     duplicates_after = duplicate_report(cleaned_for_duplicate_check)
409     print("\nDUPLICATES AFTER CLEANING")
410     display(duplicates_after)
411
412     after_cleanliness = cleanliness_report(cleaned, "After Cleaning")
413     print("Overall cleanliness after:", overall_cleanliness(cleaned), "%")
414     display(after_cleanliness)
415
416     comparison = compare_cleanliness(
417         before_cleanliness,
418         after_cleanliness
419     )
420     print("\nCLEANLINESS BEFORE/AFTER")
421     display(comparison)
422
423     plot_cleanliness(comparison)
424     plot_duplicates(duplicates_before, duplicates_after)
425
426     raw.to_csv(OUTPUT_DIR / "raw_data.csv", index=False)
427     cleaned.to_csv(OUTPUT_DIR / "cleaned_data.csv", index=False)
428     duplicates_before.to_csv(
429         OUTPUT_DIR / "duplicates_before.csv", index=False
430     )
431     duplicates_after.to_csv(
432         OUTPUT_DIR / "duplicates_after.csv", index=False
433     )
434     before_cleanliness.to_csv(
435         OUTPUT_DIR / "cleanliness_before.csv", index=False
436     )
437     after_cleanliness.to_csv(

```



```
438     OUTPUT_DIR / "cleanliness_after.csv", index=False
439 )
440 comparison.to_csv(
441     OUTPUT_DIR / "cleanliness_comparison.csv", index=False
442 )
443 audit.to_csv(
444     OUTPUT_DIR / "cleaning_audit_log.csv", index=False
445 )
446
447 print("\nSaved files:")
448 for path in sorted(OUTPUT_DIR.iterdir()):
449     print("-", path.name)
450
451 return {
452     "raw": raw,
453     "cleaned": cleaned,
454     "duplicates_before": duplicates_before,
455     "duplicates_after": duplicates_after,
456     "cleanliness_before": before_cleanliness,
457     "cleanliness_after": after_cleanliness,
458     "cleanliness_comparison": comparison,
459     "audit_log": audit
460 }
461
462
463 # Run with:
464 # results = run_project()
465
```

✓ Running full analysis of the Data

```
1 results = run_project()
```


nloaded page 1: 25,000 rows
nloaded page 2: 25,000 rows
nloaded page 3: 25,000 rows
nloaded page 4: 25,000 rows

DATA SHAPE: (100000, 12)

unique_key	created_date	closed_date	agency	complaint_type	descript
69917618	2026-08-02T02:15:47.000	NaN	DSNY	Graffiti	Gra
69922120	2026-08-02T02:06:06.000	NaN	NYPD	Noise - Street/Sidewalk	Loud Music/Pa
69922194	2026-08-02T02:05:57.000	NaN	NYPD	Noise - Street/Sidewalk	Loud Music/Pa
69924583	2026-08-02T02:05:49.000	NaN	NYPD	Noise - Residential	Banging/Poundi
69923847	2026-08-02T02:05:41.000	NaN	NYPD	Noise - Commercial	Loud Music/Pa
69918121	2026-08-02T02:05:20.000	NaN	NYPD	Noise - Street/Sidewalk	Loud Music/Pa
69915782	2026-08-02T02:05:13.000	NaN	NYPD	Noise - Residential	Loud Music/Pa
69916487	2026-08-02T02:05:12.000	NaN	NYPD	Noise - Residential	Banging/Poundi
69924519	2026-08-02T02:05:06.000	NaN	NYPD	Noise - Park	Loud Music/Pa
69918581	2026-08-02T02:04:40.000	NaN	NYPD	Noise - Residential	Loud Music/Pa

LICATES BEFORE CLEANING

Duplicate Check	Count	Meaning
Exact duplicate rows	0	All selected fields are identical
Rows with duplicate Unique Keys	0	The same service-request ID appears more than ...
Potential duplicate rows for review	2243	Same date, agency, complaint, descriptor, ZIP,...

ct duplicate examples

unique_key	created_date	closed_date	agency	complaint_type	descriptor	locat
------------	--------------	-------------	--------	----------------	------------	-------

licate Unique Key examples

unique_key	created_date	closed_date	agency	complaint_type	descriptor	locat
------------	--------------	-------------	--------	----------------	------------	-------

ential duplicate examples

unique_key	created_date	closed_date	agency	complaint_type
------------	--------------	-------------	--------	----------------